

AIML320 Assignment 2

Zade Viggers 300655355

Task 1

Part 1.1

Question a

I created a modified version of the 'generate' function in model.py that returned a python generator, which yielded after each token. This lets me pause and look at the probability distribution for each given token.

I then added a flag in sample.py to enable use of my new method and plot the distribution as it goes. When matplotlib shows a graph it makes the generation pause indefinitely, so I added a button widget to the chart which closes it and generates the next token, before showing the new distribution.

The output tokens are also streamed to the console output by calling the print function without its default trailing newline character.

I also added a shebang to the start of the sample.py file so it could be run like a shell script, rather than needing to specify python3.

Figure 1 and figure 2 show the output of this with two different prompts.

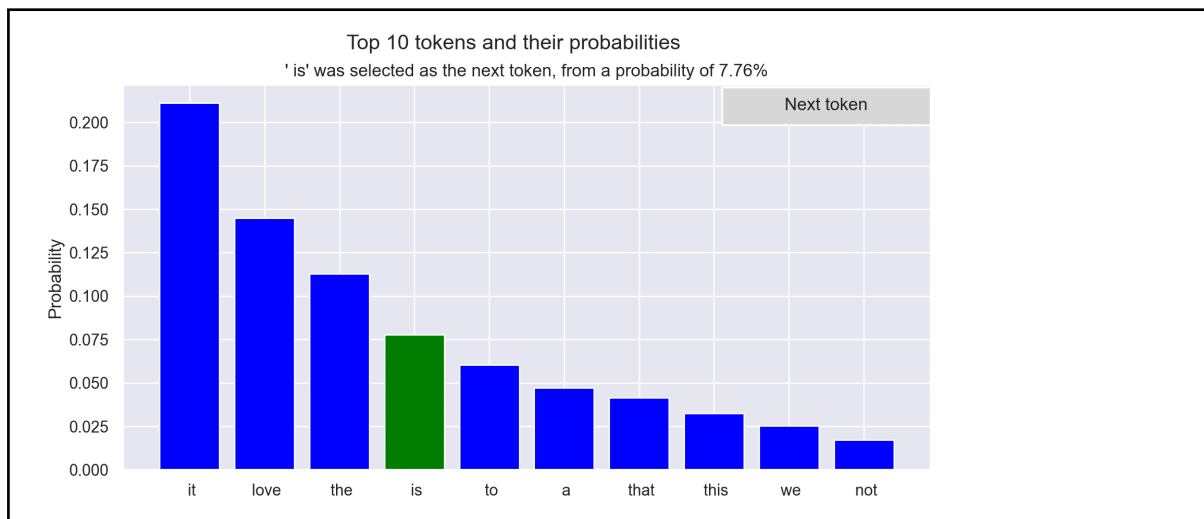
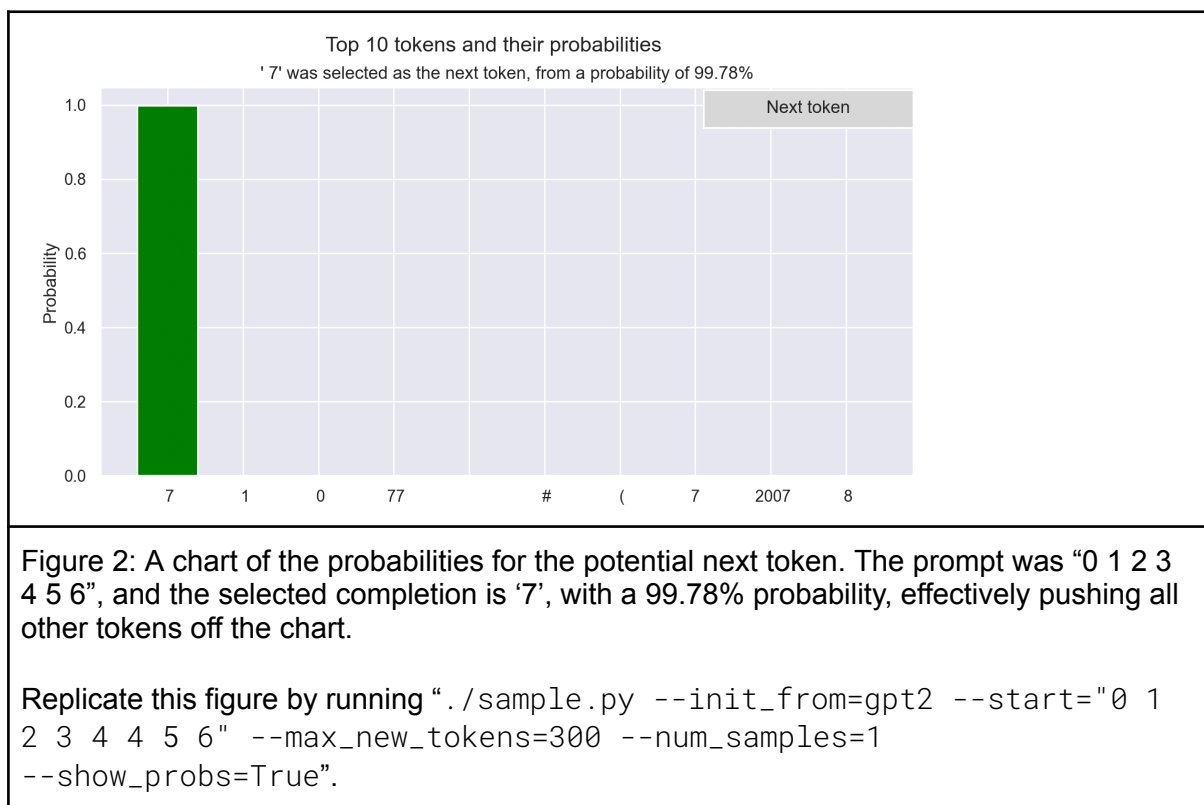


Figure 1: A chart of the probabilities for the potential next token, with the selected token 'is' highlighted in green. The initial prompt was "What is love?", and the combined completion before this token was: "What is love?\n\nLove is the result of a desire to be loved, and".

Replicate this figure by running `./sample.py --init_from=gpt2 --start="What is love?" --max_new_tokens=300 --num_samples=1 --show_probs=True` and then clicking the next token button several times.



Question b

The temperature command line argument allows for a wider range of tokens to be considered. Low temperature means most of the weight is given to one or a few tokens, while higher temperatures mean that the weight is spread around a lot more. This can be seen in figure 3.

The tokens are still selected with the same method no matter what the temperature, but the chance of any given token being selected is higher with a higher temperature. Very low temperature often results in responses getting stuck in 'loops', repeating the same phrase over and over again, while very high temperature leads to unfocused rambling compilations that might not make sense at all.

In my unstructured subjective testing, a temperature around 0.5 produces higher quality results.

Figure 3: Probability distribution of top 10 most probable tokens for different temperatures with the prompt "What is love? Baby don't"

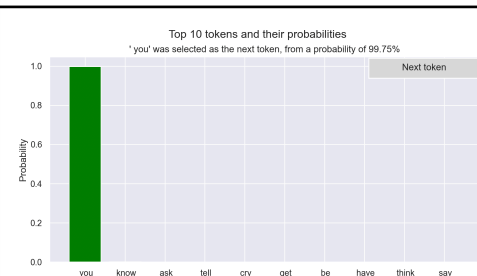


Figure 3.1: Temperature = 0.1.
The selected token has a probability of 99.75%.

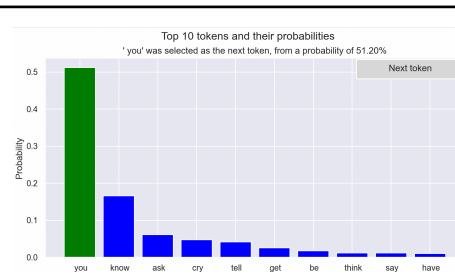


Figure 3.2: Temperature = 0.5.
The selected token has a probability of 51.20%.

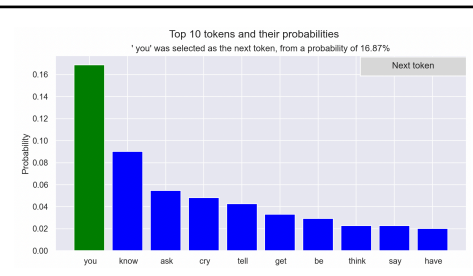


Figure 3.3: Temperature = 0.95.
The selected token has a probability of 16.87%.

Part 1.2

Question a

I adapted the existing all-at-once generation function rather than my streaming function for this part. After each token, I take the natural log of its post-softmax probability, and add that log probability to a running sum. This works because the probabilities tokens are already conditional probabilities, and adding logs of probabilities is the same as multiplying regular probabilities. Once all tokens have been generated, I exponentiate the log sum to get the probability of the whole sequence. I had to switch from *float16* to *float32* for this part, as the low temperature was causing number overflows.

The response to the prompt 'I live in' is ' a small town in the ', with a probability of 100%.

Question b

The only assumptions made here are that the model is a good estimator (i.e. it produces the conditional probabilities of tokens that represent language well), and that the identity $\log(a * b) = \log(a) + \log(b)$ holds true.

When the sequence of completion tokens being generated by the model is very long, the probability of that given sequence rapidly becomes extremely small, as is natural when multiplying percentages. This presents an issue for limited-precision representations of floating point numbers, as they can underflow. Using log probabilities is helpful for long completions because it solves the problem of floating point numbers underflowing. Logs can handle very small numbers very efficiently. They also grow large at a slower rate due to them being summed rather than multiplied.

Part 1.3

Question a

The code for this part is quite straightforward. I encode the fixed response tokens, and then at each iteration step instead of sampling stochastically, I just use the next token in the fixed response sequence. The fixed token's probability is extracted from the softmax vector the same way as if it were selected properly.

I set up a mechanism where the system will always finish the fixed response, even if it is longer than the maximum tokens parameter would allow. However if the maximum number of response tokens has not yet been reached when the fixed response tokens are exhausted, I fall back to the stochastic sampling method, allowing the user to explore the combined probability of 'natural' completions after their fixed completion. I also applied the new mechanism to my streaming tokens variant of the function, which is used when showing the probability distribution of the top 10 tokens, to make exploring the completion space to fill in Table 1 more efficient.

Shown in Table 1 are some fixed responses I tested for the prompt "I live in". One fun thing to notice are the order of magnitude differences in probability between "New Zealand", "Australia", and "New York".

fixed_response=	Probability
" a building"	0.14%
" a tree"	0.00%
" New York"	3.60%
" New Zealand"	0.03%

" Australia"	0.13%
" the middle of nowhere"	1.81%

Table 1: The probability of different responses to the prompt "I live in".
Command used: `./sample.py --init_from=gpt2 --start="I live in" --max_new_tokens=1 --num_samples=1 --temperature=0.7 --fixed_response=$Response`

Question b

To investigate what effect long sequences have on the probability, I first used a temperature of 0.0001 to get the most likely completion for several prompts with different completion lengths. Then, I set the temperature to 0.7 with those pre-determined most likely completions as the fixed response to get their probability. The results are shown in Table 2.

The probability rapidly decreases for longer response lengths, as one would expect when multiplying many percentages.

Prompt	Fixed response tokens	Probability at temperature=7
"The best"	1	5.27E-01
	10	2.46E-04
	50	2.06E-14
"I am the first to"	1	7.50E-01
	10	1.60E-03
	50	1.20E-11
"My favorite thing is"	1	4.71E-01
	10	9.64E-06
	50	3.11E-19

Table 2: The probability of the completion where the most likely token was taken each time. This was done by predicting a response of different lengths to a prompt with temperature=0.0001, then using that completion as the fixed_completion argument with temperature=0.7.
Command to get most likely completion:
Command to get probability of completion:

Task 2

Part 2.1

Question a

I took the *sample.py* code and adapted it to run in a loop over the prompt pairs read from the json file and print a nice terminal output. I removed parameters (start, fixed_response) set by the file, and changed the defaults for some others to reduce needed arguments. I was able to do this since as a new file means I won't be breaking existing APIs by changing these. I was also running into some divide by zero errors in probability calculation, due to numeric instability, so adapted the log probability to be calculated from a separate softmax using *torch.log_softmax* which uses a special formulation to avoid this problem.

Example responses for the provided file are in Table 3.

Prompt	Fixed response	Response probability
"I live in "	"New Zealand"	5.596566831192393e-10
"My favourite food is "	"pavlova"	5.596566831192393e-10

Table 3: Fixed response probabilities to the initial prompts in *eval_data.json*. Computed with this command: `./eval.py --temperature=0.7`

Question b

My program checks that it is being run as a command (by checking the value of the `__name__` variable) and not imported, then runs *eval()* with the name of the json file hardcoded. This means that the *eval.py* file can be imported without running automatically, and the program importing it can specify which json file to read.

Part 2.2

Question a

I have chosen the corpus on hugging face called "NCERT_Geography_11th" by user "KadamParth". This is a dataset of geography / earth science question and answer pairs.

Question b

I manually downloaded the CSV file from Hugging Face, and wrote a script in *convert_geography_datatset_to_json.py* to convert it to the json format automatically. I shuffle the list of rows, and then save 10 of them to the json file as the evaluation set.

Part 2.3

I adapted the shakespeare finetuning config & data prep code based on my code in *convert_geography_datatase_to_json.py* to set up the data for finetuning. [eval.py](#) was also updated to structure questions in a 'Question: XXX Answer: XXX' format, as that is what is provided in the finetuning dataset. See appendix A for the full set.

Question a

The fine tuning results are shown in Table 4. The fine tuning went acceptably, with 8 of the 10 test cases showing an improvement. The improvement ranged between less than 1 and 11 orders of magnitude, but no responses were significantly more likely - they were all still very small.

Prompt #	Probability before finetuning	Probability after finetuning	Change
1	9.662864384548772e-33	3.4917069761227404e-33	Worse
2	1.5961106932609356e-18	6.719840376171146e-18	Better
3	3.345522999901911e-14	1.5835499180403795e-15	Worse
4	6.795073745861429e-41	4.4702611854986944e-35	Better
5	1.4909667314668947e-48	3.004089476878292e-44	Better
6	3.590346283812588e-34	6.996917027750649e-33	Better
7	3.537327154458158e-17	1.5900168371530586e-16	Better
8	1.35805889178513e-77	8.393569618507681e-68	Better
9	5.422653763076162e-14	3.9332820275037856e-10	Better
10	8.400279761188711e-34	2.2125705029546027e-32	Better

Table 4: Fine-tuning results. Two are worse after fine-tuning, with the remaining 8 better after fine-tuning. 'Before' results were computed with this command: `./eval.py --temperature=0.7`
'After' results were computed by fine tuning with this command: `./train.py config/finetune_geog.py`, followed by evaluation with this command: `./eval.py --temperature=0.7 --init_from=resume --out_dir=out-geog`

Question b

One method to improve the evaluation is a single-output LLM-as-a-Judge method. The LLM would be allowed to generate an original response to the question, rather than just being judged on how likely it is to generate the exact model answer. The judge then compares the generated answer to the model answer, and assigns it a grade. This measures how 'well' the response matches the original, whilst allowing for differences in grammar and wording. It still is testing memorisation rather than intelligence though.

Another similar method is a multi-output human-in-the-loop method, where the model's answer after a training step is compared to its answer before that training step by a human. The human chooses which answer they think is better. This is used as a binary signal for whether the model has improved or not. This requires a lot of human manpower though, so would be best used in conjunction with an automated evaluation system, possibly even the same system but with an LLM-as-a-Judge instead of a human. The human would then be called in less often, but their judgments would be given significantly higher weight than the LLM judge.

Task 3

Part 3.1

Blocks are the key part of transformer models that allow them to understand long contexts and complex structures.

Each of NanoGPT's blocks consists of two parts - a multi-head self attention mechanism followed by a multi-layer perceptron. There are also normalisation layers that are applied to the input before passing it to each of these two layers (called a pre-norm architecture), in order to stop the values from exploding wildly. Normalising the inputs this way stabilises training significantly. The whole block is deceptively simple for how powerful it is. The block class code can be seen in Figure 4.

```
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = LayerNorm(config.n_embd, bias=config.bias)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config.n_embd, bias=config.bias)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```

Figure 4: NanoGPT Block implementation

The multi-head self attention is applied to the input - each head can run in parallel before the outputs are combined. In earlier blocks, these attention mechanisms might be focusing on sub-sentence level semantics, like putting a verb after a subject, while in later blocks they might focus on more abstract concepts like keeping the names and pronouns of multiple characters consistent throughout a paragraph.

The multi-head self attention is the part of the reason that later blocks are able to understand complex concepts like characters and how to refer to them. The MLP allows the model to learn additional information that it can associate with different outputs from the attention system. Because it processes each token's representation independently, it can transform the highly contextual output from the attention system to a more abstract representation. This means that the model's learned knowledge can be combined with the information about the context from the attention mechanism, allowing it to 'think'.

Another important thing to note from Figure 4 is the ' $x = x + \dots$ ' structure used when applying the attention and multi-layer perceptron to the input, which is called a residual connection. This means each block is learning to modify the input, rather than re-creating it from scratch, enabling much more efficient learning. These residual connections are quite distinctive of modern transformers.

As I have already alluded to, there are many blocks stacked one after another in a full GPT-2 model. The hyperparameter *n_layer* controls how many blocks are used - the GPT-2 model I used throughout this report used 12 blocks. You can see the blocks being created in the GPT class' `__init__` function in Figure 5.

```
class GPT(nn.Module):

    def __init__(self, config):
        ...
        self.transformer = nn.ModuleDict(dict(
            ...
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ...
        ))
        ...
    ...
    def forward(self, ...):
        ...
        for block in self.transformer.h:
            x = block(x)
        ...
    ...
    def crop_block_size(self, block_size):
        ...
        for block in self.transformer.h:
            if hasattr(block.attn, 'bias'):
                block.attn.bias = block.attn.bias[:, :, :block_size, :block_size]
        ...
```

Figure 5: NanoGPT's use of attention blocks

The way the blocks are used is truly very straight-forward (get it?), as you can see in the GPT class' *forward* method in Figure 5. They are just applied one after the other. This means that the MLP of one block is feeding directly into the attention mechanism of the next block, which is at a high level the very classic neural network architecture.

Throughout training the weights of all the attention mechanisms and the MLPs can adapt to improve the final output. Looking at the number of parameters you need to train for a small model like this, it is easy to see how state-of-the-art models can take months to train on the world's biggest computers.

To help illustrate how a block works, I've made a script called `trace.py` that applies the first part of a block one section at a time, shown in Figure 6. You can see how the normalisation perfectly centers the random data on a mean of 0 with a standard deviation of 1. Then you can see how the attention mechanism affects all the elements in the tensor.

```
$ cat tracing.py
#!/ python3

import torch
from model import Block, GPTConfig

x = torch.randn(2, 10, 768) # [batch, seq_len, embed_dim]
print(f"Input shape: {x.shape}")
print(f"Input mean: {x.mean():.4f}, std: {x.std():.4f}")

block = Block(GPTConfig())

# After LayerNorm
x_norm = block.ln_1(x)
print(f"After LN1 mean: {x_norm.mean():.4f}, std: {x_norm.std():.4f}")

# After attention + residual
x_attn = x + block.attn(x_norm)
print(f"After attention shape: {x_attn.shape}")
print(f"Residual magnitude: {(x_attn - x).abs().mean():.4f}")

$ ./tracing.py
Input shape: torch.Size([2, 10, 768])
Input mean: -0.0002, std: 0.9938
After LN1 mean: 0.0000, std: 1.0000
After attention shape: torch.Size([2, 10, 768])
Residual magnitude: 0.1384
```

Figure 6: Terminal output showing the content of `tracing.py` and the output of it tracing results through a block

Part 3.2

Question a

I have created an interactive chart that displays the attention weights of all the previous tokens in the context. By default it averages the weights of all of the heads of the final block. However, the user can also aggregate all the blocks, or select a specific block and head.

It will be useful whilst fine tuning to understand what the model is focusing on, and could help aid in manually creating additional test cases to refine fine tuning. It could also serve as a teaching aid or a simple tool for the curious user to inspect how an LLM looks at the context. My visualisation is also very helpful for understanding how the attention weights change through the blocks in the transformer, as you can scrub through them with a slider - see Figure 7.

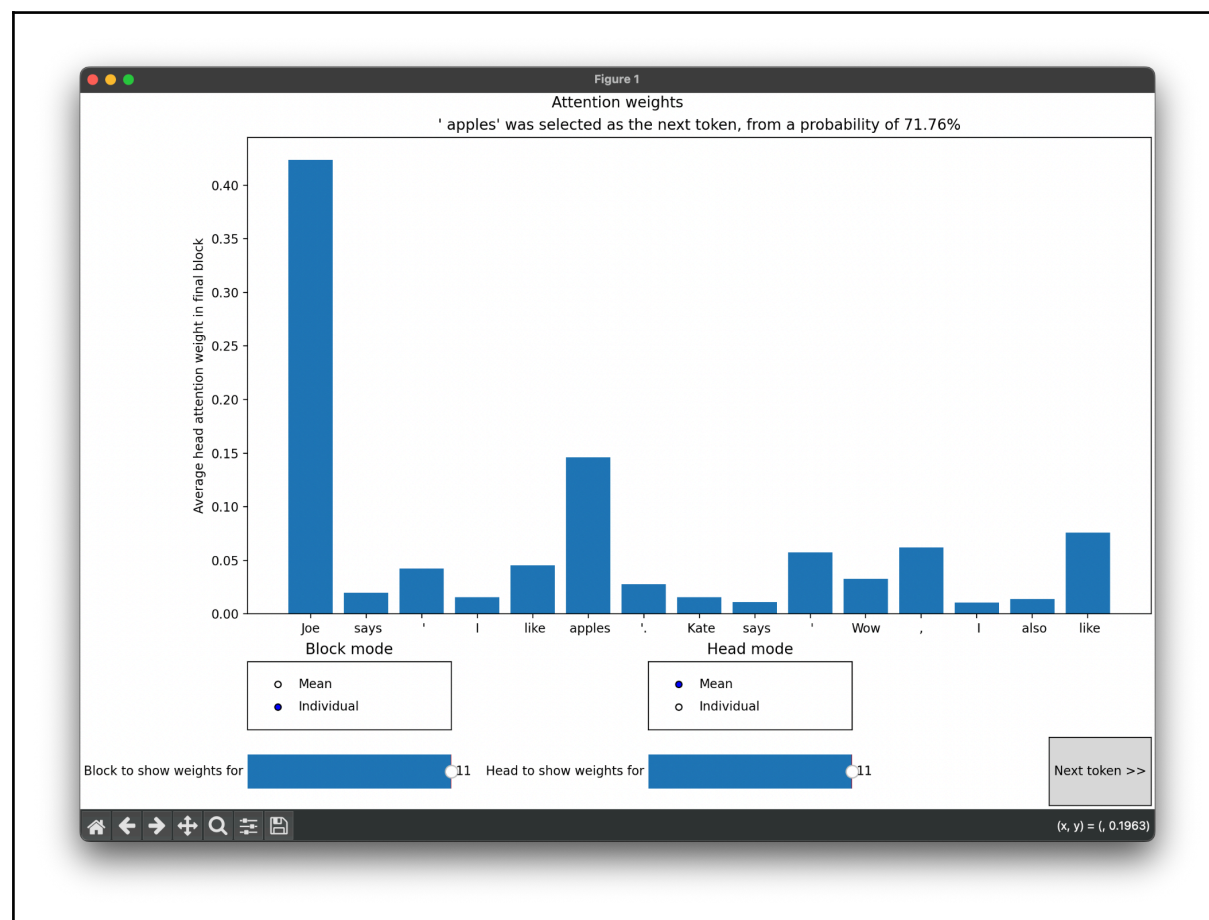


Figure 7: An interactive matplotlib chart showing attention weights on the context. For both blocks and heads, there are radio buttons to select between 'Mean' and 'Individual' modes. When in 'Individual' mode there is a slider that can be used to choose which head or block to inspect.

Generated with this command: `./sample.py --init_from="gpt2" --num_samples=1 --max_new_tokens=10 --start="Joe says 'I like apples'. Kate says 'Wow, I also like"` --show_attention=True

In order to implement it, I needed to snapshot attention weights while predicting. I added an extra config flag that goes all the way down to the attention mechanism to disable flash (hardware optimised) attention so I can access weights directly. I then passed attention weights all the way back up and saved them onto the GPT class.

For the UI, I built on top of my streaming token system, adding an extra flag called *show_attention*. I built on my knowledge from the charts in Part 1.1 to add more interactive features, like sliders and radio buttons to the matplotlib chart. You can see these in Figure 7.

Question b

It's interesting to scrub through the individual heads in a given block, and see the diversity of focus amongst them. In figure 8 you can see that within the final block there are still many different tokens that the attention heads are looking at.

Figure 8: Inspecting different attention heads.

Generated with this command: `./sample.py --init_from="gpt2" --num_samples=1 --max_new_tokens=10 --start="My favorite ice cream flavour is" --show_attention=True`

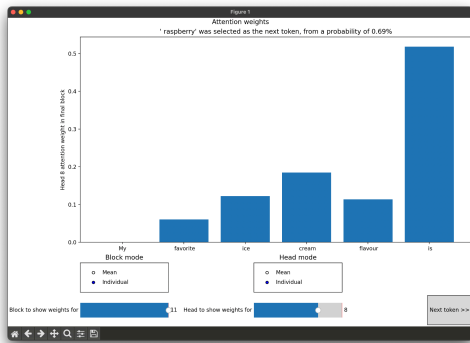


Figure 8.1: left skewed attention head

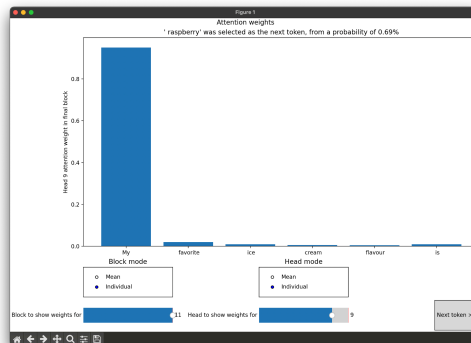


Figure 8.2: right skewed attention head

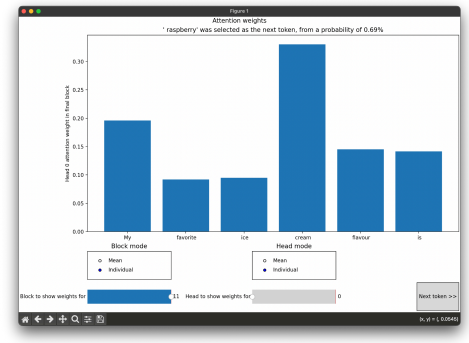


Figure 8.3: somewhat uniform attention head.

I also found it quite cool scrubbing through the block stack. In Figure 9 you can observe how the weights start quite spread out but as you get to the end of the blocks they get more concentrated.

Figure 9: Investigating how the attention head weights change throughout the block stack.

Generated with this command: `./sample.py --init_from="gpt2" --num_samples=1 --max_new_tokens=10 --start="You come to me on the day of my daughter's" --show_attention=True`

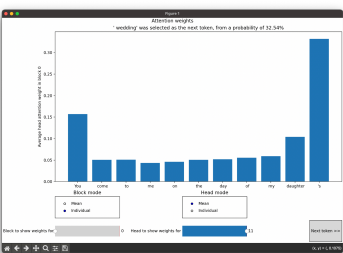


Figure 9.1: Wide distribution at block 0

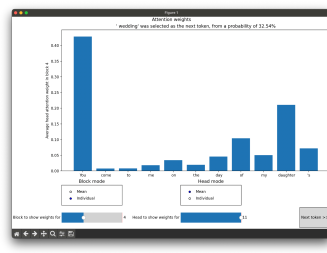


Figure 9.2: More focused on 'You', but still somewhat diverse at block 4

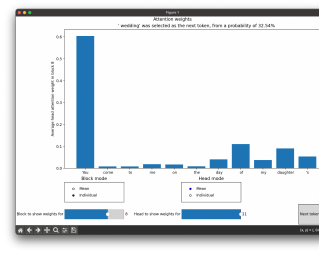


Figure 9.3: Mostly focused on 'You', but with a little bit of noise to the right at block 8

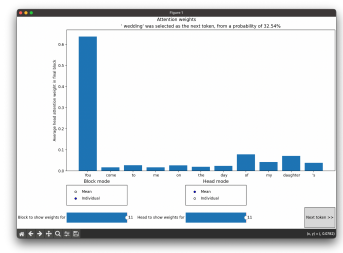


Figure 9.4: Similar to block 8, block 11 is mostly focused on 'You', with slightly less noise on the right.

Appendices

Appendix A: Automatically selected eval_data.json prompts and responses

The format “*Question: \$PROMPT\nAnswer:* ” is used as the prompt for the model, where “\$PROMPT” is substituted for the value in the prompt column below.

Number	Prompt	Response
1	Describe the climate conditions in the 1990s.	The 1990s recorded the warmest temperatures of the century and experienced some of the worst floods around the world.
2	What are reservoir induced earthquakes?	Reservoir induced earthquakes are earthquakes that occur in areas where large reservoirs are located.
3	What is the process called where rocks and minerals break down due to exposure to the atmosphere, water, and biological organisms?	Weathering
4	Describe the role of climate in soil formation.	Climate influences soil formation through temperature and moisture. Temperature affects the rate of chemical reactions and biological activity, while moisture influences the movement of materials and the growth of plants and microorganisms.
5	Provide an example of a satellite and its sensor, along with its spatial resolution.	An example of a satellite and its sensor is the Landsat TM (Thematic Mapper) from the USA, which has a spatial resolution of 30.00 × 30.00 meters.
6	Provide an example of a natural process that contributes to soil formation.	An example of a natural process that contributes to soil formation is the weathering of rocks, where physical and chemical processes break down rocks into smaller mineral particles.
7	What is the term for the network of parallels and meridians?	Graticule
8	Provide an example of how the annual range of temperature might affect a region's climate.	The annual range of temperature can significantly affect a region's climate by determining the

		extremes of heat and cold experienced throughout the year. For example, a region with a high annual range might experience very hot summers and very cold winters, leading to a continental climate. This can influence vegetation types, wildlife habitats, and human activities such as agriculture and tourism.
9	What are the two main types of events that can lead to slope instability?	The two main types of events that can lead to slope instability are natural events and human activities.
10	Provide an example of a location with its latitude and longitude.	An example is New Delhi, India, which is located at approximately 28.61 degrees North latitude and 77.23 degrees East longitude.

Appendix B: AI Use

- AI tools were used for online research, as a supplement to traditional search engines, throughout this assignment
- AI tools were used for to gain feedback (acting like a marker) when I was writing my report for Part 3.1